

Ames Housing Analysis

Ames Housing Analysis

Load Library and Inspect Data

We see that there are 74 predictors. With this many predictors, a standard Linear Regression may overfit and multicollinearity may occur. Regularized Regression will be used instead of account for this.

The Sale Price is also skewed right, most likely due to very expensive houses in the dataset. This data will need to be normalized.

```
library(tidymodels)
library(tidyverse)
library(modeldata)

# Load the built-in Ames housing dataset
data("ames")

# Check dataset size
glimpse(ames)
```

Rows: 2,930

Columns: 74

\$ MS_SubClass	<fct> One_Story_1946_and_Newer_All_Styles, One_Story_1946~
\$ MS_Zoning	<fct> Residential_Low_Density, Residential_High_Density, ~
\$ Lot_Frontage	<dbl> 141, 80, 81, 93, 74, 78, 41, 43, 39, 60, 75, 0, 63,~
\$ Lot_Area	<int> 31770, 11622, 14267, 11160, 13830, 9978, 4920, 5005~
\$ Street	<fct> Pave, Pave, Pave, Pave, Pave, Pave, Pave, Pave, Pav~
\$ Alley	<fct> No_Alley_Access, No_Alley_Access, No_Alley_Access, ~
\$ Lot_Shape	<fct> Slightly_Irregular, Regular, Slightly_Irregular, Re~
\$ Land_Contour	<fct> Lvl, Lvl, Lvl, Lvl, Lvl, Lvl, Lvl, HLS, Lvl, Lvl, L~
\$ Utilities	<fct> AllPub, AllPub, AllPub, AllPub, AllPub, AllPub, All~

\$ Lot_Config	<fct> Corner, Inside, Corner, Corner, Inside, Inside, Ins~
\$ Land_Slope	<fct> Gtl, Gtl, Gtl, Gtl, Gtl, Gtl, Gtl, Gtl, Gtl, Gtl, G~
\$ Neighborhood	<fct> North_Ames, North_Ames, North_Ames, North_Ames, Gil~
\$ Condition_1	<fct> Norm, Feedr, Norm, Norm, Norm, Norm, Norm, Norm, No~
\$ Condition_2	<fct> Norm, Norm, Norm, Norm, Norm, Norm, Norm, Norm, Nor~
\$ Bldg_Type	<fct> OneFam, OneFam, OneFam, OneFam, OneFam, OneFam, Twn~
\$ House_Style	<fct> One_Story, One_Story, One_Story, One_Story, Two_Sto~
\$ Overall_Cond	<fct> Average, Above_Average, Above_Average, Average, Ave~
\$ Year_Built	<int> 1960, 1961, 1958, 1968, 1997, 1998, 2001, 1992, 199~
\$ Year_Remod_Add	<int> 1960, 1961, 1958, 1968, 1998, 1998, 2001, 1992, 199~
\$ Roof_Style	<fct> Hip, Gable, Hip, Hip, Gable, Gable, Gable, Gable, G~
\$ Roof_Matl	<fct> CompShg, CompShg, CompShg, CompShg, CompShg, CompSh~
\$ Exterior_1st	<fct> BrkFace, VinylSd, Wd Sdng, BrkFace, VinylSd, VinylS~
\$ Exterior_2nd	<fct> Plywood, VinylSd, Wd Sdng, BrkFace, VinylSd, VinylS~
\$ Mas_Vnr_Type	<fct> Stone, None, BrkFace, None, None, BrkFace, None, No~
\$ Mas_Vnr_Area	<dbl> 112, 0, 108, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 6~
\$ Exter_Cond	<fct> Typical, Typical, Typical, Typical, Typical, Typica~
\$ Foundation	<fct> CBlock, CBlock, CBlock, CBlock, PConc, PConc, PConc~
\$ Bsmt_Cond	<fct> Good, Typical, Typical, Typical, Typical, Typical, ~
\$ Bsmt_Exposure	<fct> Gd, No, No, No, No, No, Mn, No, No, No, No, No, No, ~
\$ BsmtFin_Type_1	<fct> BLQ, Rec, ALQ, ALQ, GLQ, GLQ, GLQ, ALQ, GLQ, Unf, U~
\$ BsmtFin_SF_1	<dbl> 2, 6, 1, 1, 3, 3, 3, 1, 3, 7, 7, 1, 7, 3, 3, 1, 3, ~
\$ BsmtFin_Type_2	<fct> Unf, LwQ, Unf, Unf, Unf, Unf, Unf, Unf, Unf, Unf, U~
\$ BsmtFin_SF_2	<dbl> 0, 144, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1120, 0~
\$ Bsmt_Unf_SF	<dbl> 441, 270, 406, 1045, 137, 324, 722, 1017, 415, 994, ~
\$ Total_Bsmt_SF	<dbl> 1080, 882, 1329, 2110, 928, 926, 1338, 1280, 1595, ~
\$ Heating	<fct> GasA, GasA, GasA, GasA, GasA, GasA, GasA, GasA, Gas~
\$ Heating_QC	<fct> Fair, Typical, Typical, Excellent, Good, Excellent, ~
\$ Central_Air	<fct> Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, Y, ~
\$ Electrical	<fct> SBrkr, SBrkr, SBrkr, SBrkr, SBrkr, SBrkr, SBrkr, SB~
\$ First_Flr_SF	<int> 1656, 896, 1329, 2110, 928, 926, 1338, 1280, 1616, ~
\$ Second_Flr_SF	<int> 0, 0, 0, 0, 701, 678, 0, 0, 0, 776, 892, 0, 676, 0, ~
\$ Gr_Liv_Area	<int> 1656, 896, 1329, 2110, 1629, 1604, 1338, 1280, 1616~
\$ Bsmt_Full_Bath	<dbl> 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, ~
\$ Bsmt_Half_Bath	<dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
\$ Full_Bath	<int> 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 3, 2, ~
\$ Half_Bath	<int> 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, ~
\$ Bedroom_AbvGr	<int> 3, 2, 3, 3, 3, 3, 2, 2, 2, 3, 3, 3, 3, 2, 1, 4, 4, ~
\$ Kitchen_AbvGr	<int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
\$ TotRms_AbvGrd	<int> 7, 5, 6, 8, 6, 7, 6, 5, 5, 7, 7, 6, 7, 5, 4, 12, 8, ~
\$ Functional	<fct> Typ, Typ, Typ, Typ, Typ, Typ, Typ, Typ, Typ, Typ, T~
\$ Fireplaces	<int> 2, 0, 0, 2, 1, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, ~
\$ Garage_Type	<fct> Attchd, Attchd, Attchd, Attchd, Attchd, Attchd, Att~

```

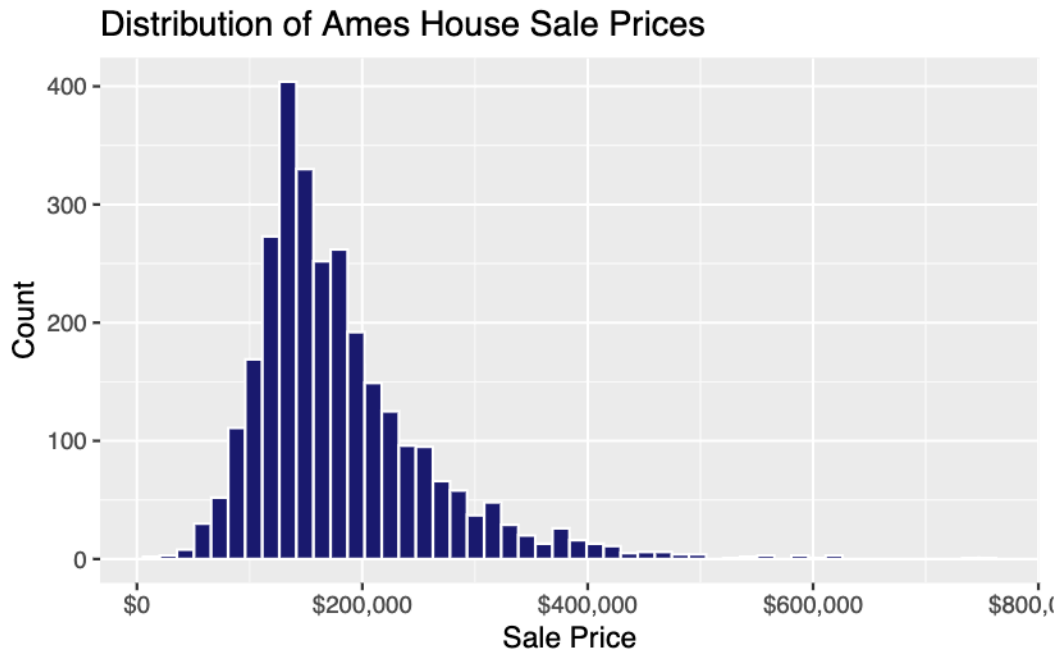
$ Garage_Finish      <fct> Fin, Unf, Unf, Fin, Fin, Fin, Fin, RFn, RFn, Fin, F~
$ Garage_Cars        <dbl> 2, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 3, 2, ~
$ Garage_Area        <dbl> 528, 730, 312, 522, 482, 470, 582, 506, 608, 442, 4~
$ Garage_Cond        <fct> Typical, Typical, Typical, Typical, Typical, Typica~
$ Paved_Drive        <fct> Partial_Pavement, Paved, Paved, Paved, Paved, Paved~
$ Wood_Deck_SF       <int> 210, 140, 393, 0, 212, 360, 0, 0, 237, 140, 157, 48~
$ Open_Porch_SF      <int> 62, 0, 36, 0, 34, 36, 0, 82, 152, 60, 84, 21, 75, 0~
$ Enclosed_Porch     <int> 0, 0, 0, 0, 0, 0, 170, 0, 0, 0, 0, 0, 0, 0, 0~
$ Three_season_porch <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ Screen_Porch       <int> 0, 120, 0, 0, 0, 0, 144, 0, 0, 0, 0, 0, 0, 140, ~
$ Pool_Area          <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ Pool_QC            <fct> No_Pool, No_Pool, No_Pool, No_Pool, No_Pool, No_Poo~
$ Fence              <fct> No_Fence, Minimum_Privacy, No_Fence, No_Fence, Mini~
$ Misc_Feature       <fct> None, None, Gar2, None, None, None, None, None, Non~
$ Misc_Val           <int> 0, 0, 12500, 0, 0, 0, 0, 0, 0, 0, 500, 0, 0, 0, ~
$ Mo_Sold            <int> 5, 6, 6, 4, 3, 6, 4, 1, 3, 6, 4, 3, 5, 2, 6, 6, ~
$ Year_Sold          <int> 2010, 2010, 2010, 2010, 2010, 2010, 2010, 2010, 201~
$ Sale_Type          <fct> WD , WD , WD , WD , WD , WD , WD , WD , WD , WD , W~
$ Sale_Condition     <fct> Normal, Normal, Normal, Normal, Normal, Normal, Nor~
$ Sale_Price         <int> 215000, 105000, 172000, 244000, 189900, 195500, 213~
$ Longitude          <dbl> -93.61975, -93.61976, -93.61939, -93.61732, -93.638~
$ Latitude           <dbl> 42.05403, 42.05301, 42.05266, 42.05125, 42.06090, 4~

```

```

# Preview target distribution
ggplot(ames, aes(x = Sale_Price)) +
  geom_histogram(bins = 50, fill = "midnightblue", color = "white") +
  scale_x_continuous(labels = scales::dollar_format()) +
  labs(title = "Distribution of Ames House Sale Prices",
       x = "Sale Price", y = "Count")

```



Data splitting and Resampling

I split the data into 80/20 train/test. Because the pricing is skewed right (with some really expensive homes in the dataset), we will stratify by sale price to prevent to ensure that a disproportionate number of those high end homes don't end up entirely in the test or train set.

```
set.seed(123)

# Split into Training (80%) and Testing (20%) sets
ames_split <- initial_split(ames, prop = 0.80, strata = Sale_Price)
train_data <- training(ames_split)
test_data <- testing(ames_split)

# Create Cross-Validation folds from the training data to evaluate models safely
ames_folds <- vfold_cv(train_data, v = 5, strata = Sale_Price)
```

Feature Engineering

Use recipes to clean and transform the data

```
ames_recipe <- recipe(Sale_Price ~ ., data = train_data) %>%
  #Log-transform the target variable to satisfy linearity assumptions
  step_log(Sale_Price, base = 10) %>%
```

```

#Pool rare categorical values (neighborhoods with <5% frequency get grouped into "Other")
step_other(all_nominal_predictors(), threshold = 0.05) %>%

#Create dummy variables for all categorical columns
step_dummy(all_nominal_predictors()) %>%

#Remove columns with near-zero variance
step_zv(all_predictors()) %>%

#Center and scale all numeric predictors
step_normalize(all_numeric_predictors())

```

Model Specifications and Workflows

Define the exact mathematical framework of the algorithm and creating a structural container to hold the entire pipeline.

```

# Define the regularized model specification
elnet_spec <- linear_reg(
  penalty = tune(),
  mixture = tune()
) %>%
  set_engine("glmnet") %>%
  set_mode("regression")

# Bundle the recipe and model into a pipeline workflow
elnet_workflow <- workflow() %>%
  add_recipe(ames_recipe) %>% #passes data through recipe defined in previous step
  add_model(elnet_spec) # takes the cleaned data and feeds it into the regularized regression

```

Hyperparameter Grid Definition and Model Fitting

```

# Create a custom grid to evaluate Lasso, Ridge, and a 50/50 mix across various penalty strengths
custom_grid <- expand_grid(
  penalty = c(0.001, 0.01, 0.1),
  mixture = c(0, 0.5, 1)
)

# Fit the models across all 5 cross-validation folds for every combination in the grid

tune_results <- tune_grid(

```

```

elnet_workflow,
resamples = ames_folds,
grid = custom_grid,
metrics = metric_set(rmse, rsq)
)

```

Compare Model Performance

Looks like Lasso is the best performing model (mixture = 1).

Lasso's victory proves that the Ames housing dataset suffers from heavy feature redundancy and noise. By completely dropping unhelpful and collinear features to zero, Lasso simplified the network, lowered the model's variance, and generalized better to testing data.

```

# View performance of all combinations sorted by lowest RMSE
show_best(tune_results, metric = "rmse")

```

```

# A tibble: 5 x 8
  penalty mixture .metric .estimator   mean     n std_err .config
  <dbl>   <dbl> <chr>   <chr>     <dbl> <int>   <dbl> <chr>
1  0.001     1 rmse standard  0.0690     5 0.00570 pre0_mod3_post0
2  0.001     0.5 rmse standard  0.0691     5 0.00566 pre0_mod2_post0
3  0.001     0 rmse standard  0.0691     5 0.00558 pre0_mod1_post0
4  0.01      0 rmse standard  0.0691     5 0.00558 pre0_mod4_post0
5  0.1       0 rmse standard  0.0700     5 0.00486 pre0_mod7_post0

```

```

# Automatically isolate the single best performing model setup
best_model_params <- select_best(tune_results, metric = "rmse")
print(best_model_params)

```

```

# A tibble: 1 x 3
  penalty mixture .config
  <dbl>   <dbl> <chr>
1  0.001     1 pre0_mod3_post0

```

Final Fit and Test Set Evaluation

The R2 value is around 0.88, meaning the model successfully explains 88% of the variance in Ames housing prices. Only 12% of the variance is driven by unmeasured factors.

```

# Finalize the workflow
final_workflow <- finalize_workflow(elnet_workflow, best_model_params)

# Execute the final fit on the train/test split
final_fit <- last_fit(final_workflow, split = ames_split)

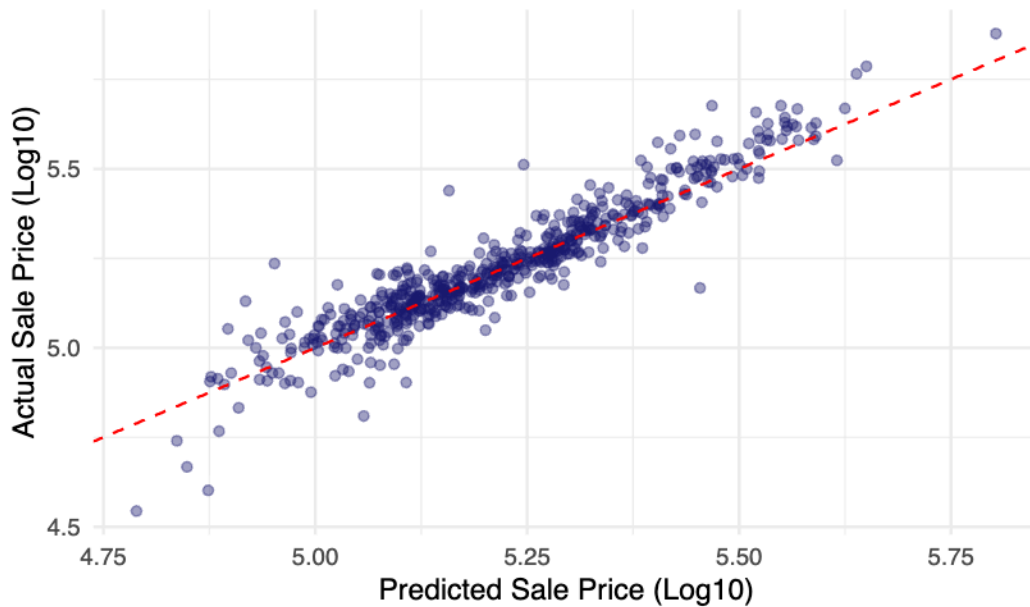
# Output the metrics on test dataset
print(collect_metrics(final_fit))

# A tibble: 2 x 4
  .metric .estimator .estimate .config
  <chr>   <chr>       <dbl> <chr>
1 rmse    standard      0.0613 pre0_mod0_post0
2 rsq     standard      0.888  pre0_mod0_post0

# Plot Predicted vs. Actual values to visually assess performance
collect_predictions(final_fit) %>%
  ggplot(aes(x = .pred, y = Sale_Price)) +
  geom_point(alpha = 0.4, color = "midnightblue") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(
    title = "Actual vs. Predicted House Prices",
    x = "Predicted Sale Price (Log10)",
    y = "Actual Sale Price (Log10)"
  ) +
  theme_minimal()

```

Actual vs. Predicted House Prices



```
# Extract RMSE
test_rmse <- final_fit %>%
  collect_metrics() %>%
  filter(.metric == "rmse") %>%
  pull(.estimate)

# Extract R-squared
test_rsqr <- final_fit %>%
  collect_metrics() %>%
  filter(.metric == "rsqr") %>%
  pull(.estimate)

cat("RMSE:", round(test_rmse, 4), "\n")
```

RMSE: 0.0613

```
cat("R2: ", round(test_rsqr, 4), "\n")
```

R2: 0.8876

Because the recipe applied a log10 transformation to the Sale_Price, final test set RMSE is measured in log-units, not dollars.

After backing out the `log10()` transformation, a RMSE of 0.0613 equates to 15.2% average error margin.

```
pct_error <- (10^test_rmse - 1) * 100

cat("On average, the model's predictions are within +/-", round(pct_error, 1), "% of the acti
```

On average, the model's predictions are within +/- 15.2 % of the actual home value.

```
# Extract the trained model engine and tidy the coefficients
model_coefficients <- final_fit %>%
  extract_fit_parsnip() %>%
  tidy()
```

Warning: package 'glmnet' was built under R version 4.5.2

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Loaded glmnet 5.0

```
# Filter out the intercept and any features that Lasso zeroed out
active_features <- model_coefficients %>%
  filter(term != "(Intercept)", estimate != 0) %>%
  arrange(desc(abs(estimate)))
```

```
# Print the top 10 most influential features
print(head(active_features, 10))
```

```
# A tibble: 10 x 3
  term                estimate penalty
  <chr>              <dbl>    <dbl>
1 Gr_Liv_Area        0.0609    0.001
2 Garage_Cars        0.0196    0.001
3 Year_Remod_Add     0.0184    0.001
```

4	Neighborhood_Somerset	0.0153	0.001
5	Sale_Condition_Normal	0.0150	0.001
6	Sale_Condition_Partial	0.0133	0.001
7	Fireplaces	0.0130	0.001
8	Year_Built	0.0127	0.001
9	Neighborhood_Northridge_Heights	0.0120	0.001
10	MS_Zoning_Residential_Medium_Density	-0.0112	0.001

GR_Liv_Area (Above-ground square footage) the largest positive coefficients, meaning physical size and is a dominant lever for property valuation in this market.